

代码管理工具Git

1. 简介

a. Git发展简史

2. 安装，在官网下载安装

3. Git代码与相关细节

a. Git 核心概念

b. Git中文件不同标识状态含义

c. 加入仓库的步骤与相关命令语句

4. 快速体验

5. 提交至远程仓库

i. 注册码云账号

ii. SSH配置

iii. 推送到远程仓库

iv. 从远程仓库克隆

6. 实际开发项目经验——多人协作开发

a. 小组成员本地开发

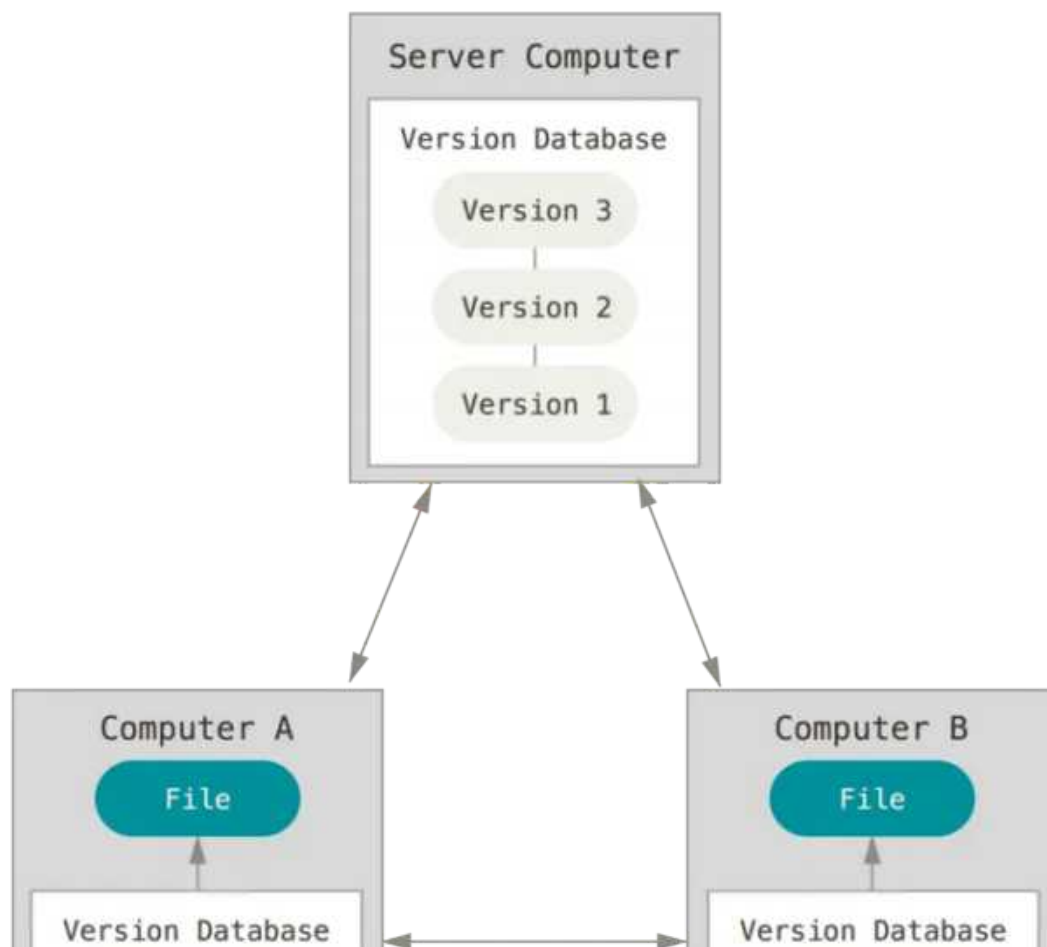
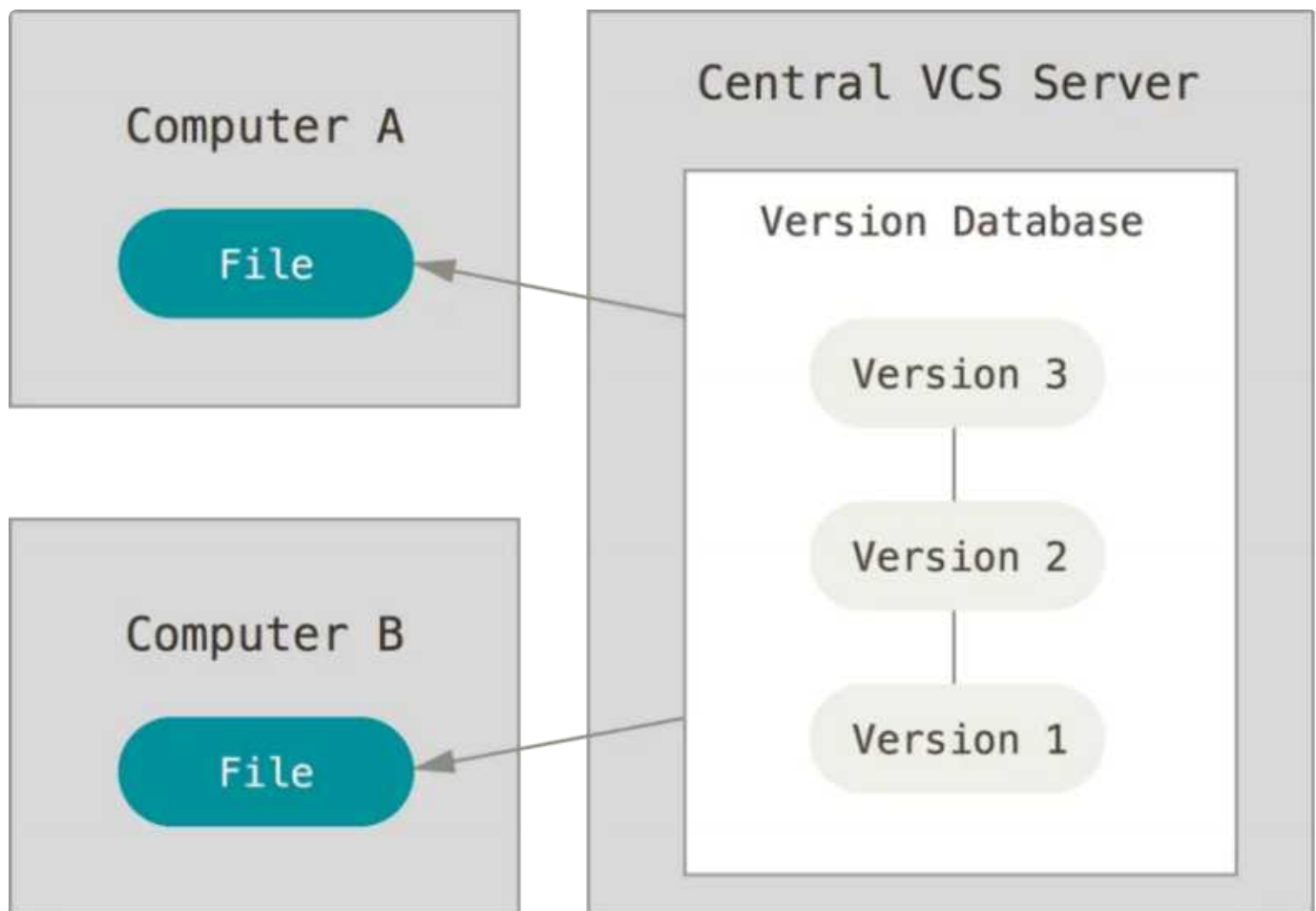
b. 组长

1. 简介

许多人习惯用复制整个项目目录的方式来保存不同的版本，或许还会改名加上备份时间以示区别。这么做特别容易犯错

为了解决这个问题，人们很久以前就开发了许多种本地版本控制系统，大多都是采用某种简单的数据库来记录文件的历次更新差异

两种主流的代码版本管理方式：集中式与分布式





在一些领域，如政府军工等仍然使用集中式，不易泄密，但是中心服务器压力很大。华为目前仍然是采用这种方式，典型项目：SVN

分布式，需要对分支节点有理解。代码仓库分布在各个节点，某一节点的问题不会影响主分支，但是不同分支的同步往往比较麻烦。

a. Git发展简史

Linux 内核开源项目有着为数众广的参与者。绝大多数的 Linux 内核维护工作都花在了提交补丁和保存归档的繁琐事务上（1991–2002年间）。到 2002 年，整个项目组开始启用一个专有的分布式版本控制系统 BitKeeper 来管理和维护代码。到了 2005 年，开发 BitKeeper 的商业公司同 Linux 内核开源社区的合作关系结束，他们收回了 Linux 内核社区免费使用 BitKeeper 的权力。这就迫使 Linux 开源社区（特别是 Linux 的缔造者 Linus Torvalds）基于使用 BitKeeper 时的经验教训，开发出自己的版本系统

2. 安装，在官网下载安装



git

[Githttps://git-scm.com/](https://git-scm.com/)



参考资料

1猴子都能懂的GIT入门 | 贝格乐 (Backlog)

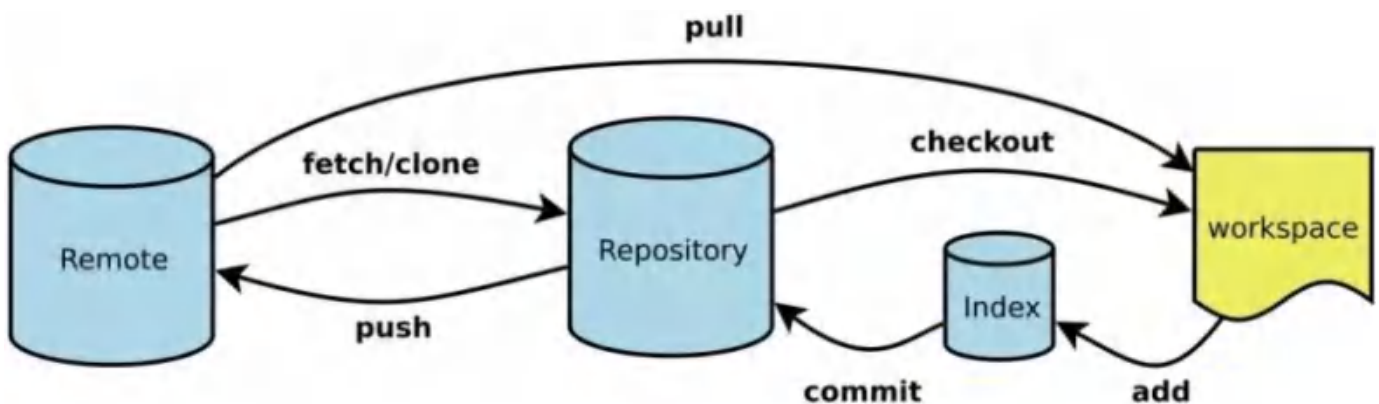
2Learn Git Branching

3Pro Git 中文版 (第二版)

4GitHub 漫游指南

3. Git代码与相关细节

a. Git 核心概念



4个主要区域：工作区、暂存区、本地仓库、远程仓库

如果用git fetch纯命令行 vim的用法，需要掌握Linux

b. Git中文件不同标识状态含义

c. 加入仓库的步骤与相关命令语句

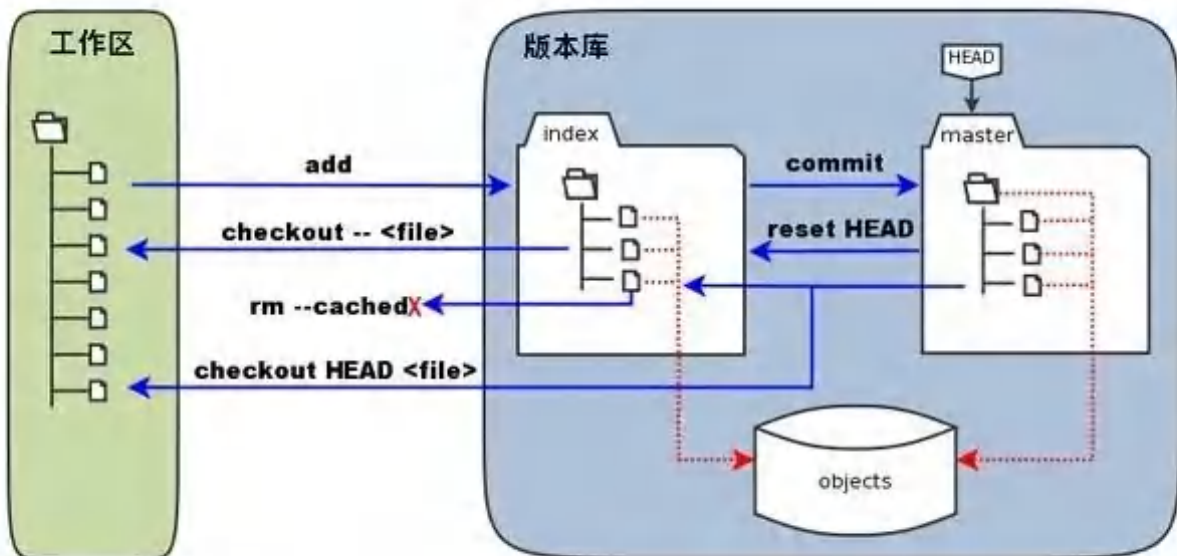
在创建代码之后：

①git init Git初始化，会在当前目录，创建一个隐藏.git文件夹

②git add . 添加到暂存区

③git commit -m'版本名字' 提交到本地仓库

※git log 可以查看版本日志（版本号是一个哈希串）



④git remote add origin HTTP/SSH协议地址

⑤git push -u origin "master/分支名" 推到远程仓库（远程仓库分支名尽量与本地仓库一致）

※远程仓库配置与各种操作在李老师的笔记里再看

与分支相关的一些命令操作

创建分支实际上就是创建一个指针指向某个commit节点，切换分支就是改变HEAD指针，使其指向新的分支。每一次提交形成节点

①git branch (-a) 显示所有的分支

②git branch name 创建一个新的分支

③git switch name

git checkout name 切换一分支

④git branch -d name 删除一个分支

⑤git merge name 合并分支到主分支

⑥git pull (origin name) 将分支拉到本地

一些可能会用到的命令

git status

git restore --staged d.html 还原暂存区的修改

git restore d.html 再还原一次，将工作区的改动退回

git remote -v

gitignore (node_module一般我们不传到远程仓库上)

4. 快速体验

1) 初始化

- 该命令会在当前目录, 创建一个隐藏的文件夹.git(就是本地仓库)

2) 创建文件

在根目录下, 创建a.html



- u: 未追踪的状态

3) 管理文件

- 添加到暂存区
- 提交到本地仓库

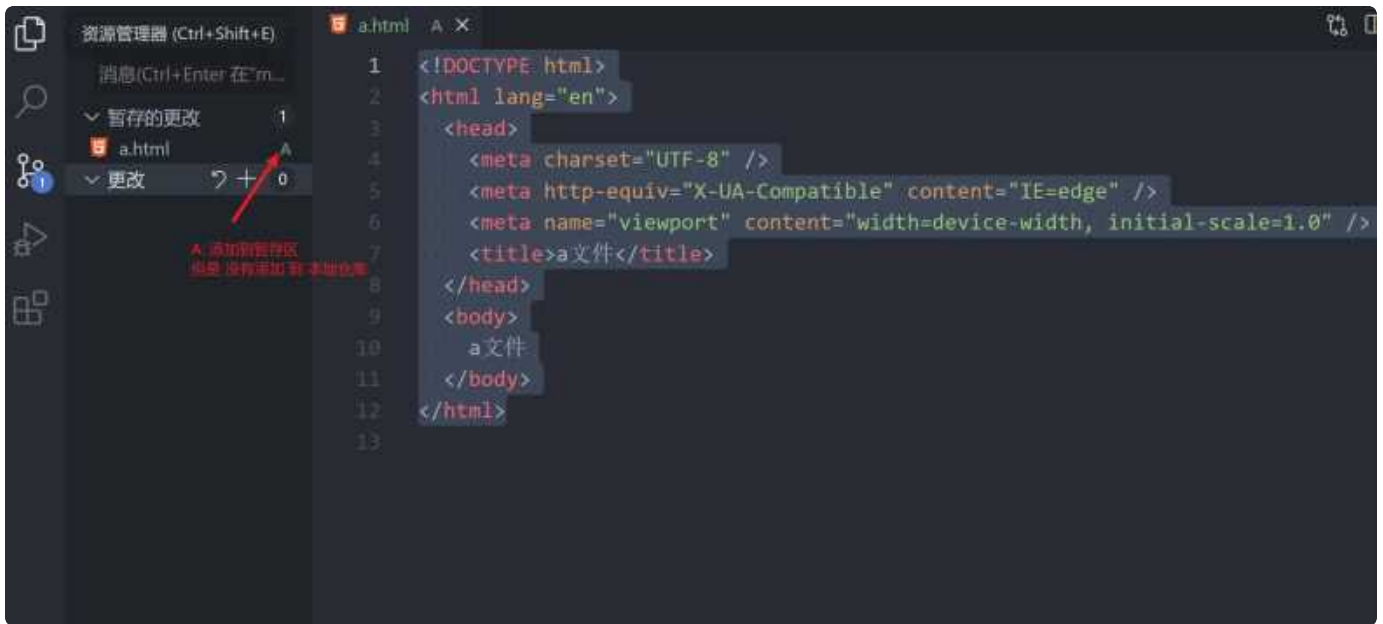
添加到暂存区

点击更改处的+, 将所有未跟踪的文件 添加 到 暂存区

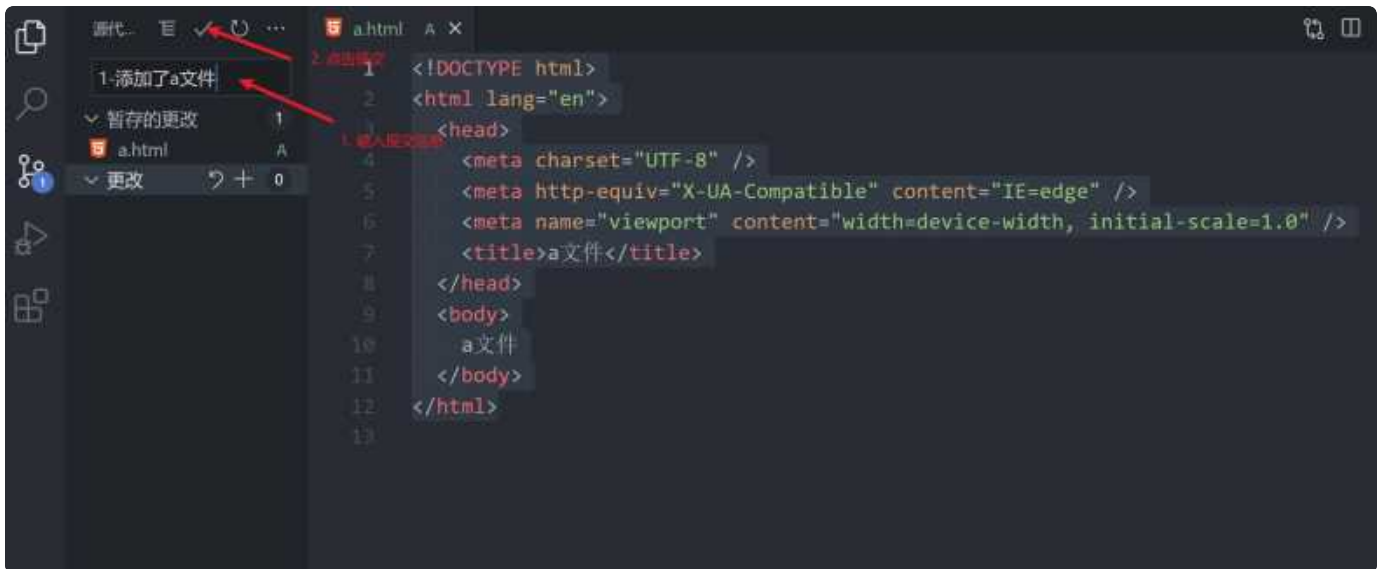


也可以通过点击单个文件后面的+, 把该文件添加到暂存区

此时, 文件的状态 变成 A



提交到本地仓库



注意

在第一次提交时, 需要 配置 user.name和 user.email

示例

在用户的根目录C:\Users\默认用户下, 写全局配置文件

```
.gitconfig ×
1  [user]
2      email = jj112358@163.com
3      name = brojie
4
```

此处仍剩余提交至远程仓库未整理

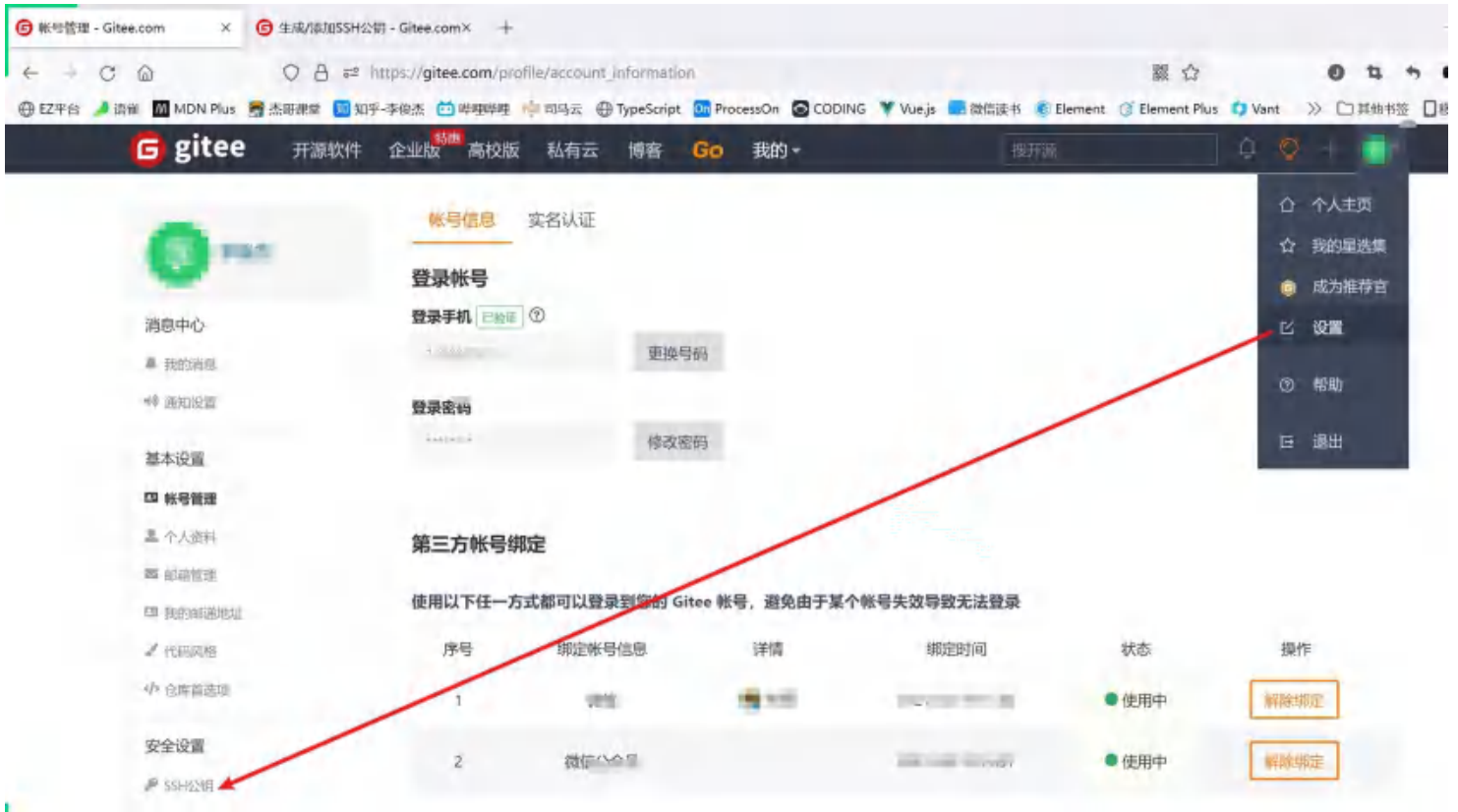
5. 提交至远程仓库

i. 注册码云账号

 [Gitee](#) – 基于 Git 的代码托管和研发协作平台

ii. SSH配置

在 [Gitee](#) 的个人中心, 找到 [设置->SSH公钥](#) 配置



点击 [怎样生成公钥](#) 查看文档

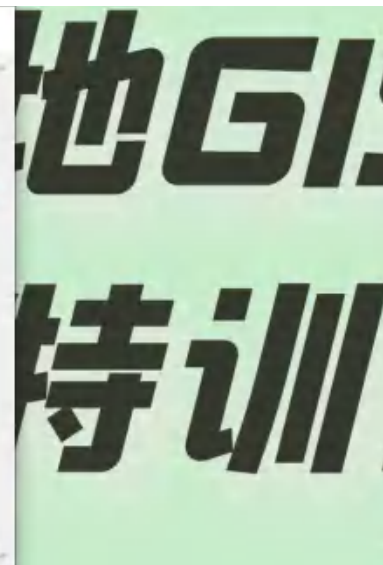


打开终端, 执行 `ssh-keygen -t ed25519 -C "xxxxxx@xxxxxx.com"` 命令
在这里xxx输入自己的邮箱

```

MINGW64:/c/Users/lijun/Desktop
lijunjie@DESKTOP-K6RFVC1 MINGW64 ~/Desktop
$ ssh-keygen -t ed25519 -c "jj112358@163.com" 1.输入自己的邮箱
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/lijun/.ssh/id_ed25519): |
2. 公钥/私钥保存的路径

```



```

lijunjie@DESKTOP-K6RFVC1 MINGW64 ~/Desktop
$ ssh-keygen -t ed25519 -c "jj112358@163.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/lijun/.ssh/id_ed25519):
Created directory '/c/Users/lijun/.ssh'.
Enter passphrase (empty for no passphrase): 按两次回车, 不需要passphrase
Enter same passphrase again:

```

```

Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/lijun/.ssh/id_ed25519): |
Created directory '/c/Users/lijun/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/lijun/.ssh/id_ed25519
Your public key has been saved in /c/Users/lijun/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:tgCxxqlQVxyv21fUWy8g4OA1+nw1WuYNTqoYrj9xpA8 jj112358@163.com
The key's randomart image is:

```

```

+--[ED25519 256]--+
| ..oo. |
| . o oo = |
| . + o. * + + |
| .. . .o o + X . . |
| . . +So % + o. |
| Eot+.o + + . |
| . X.. - . |
| = + . |
| .o.. . |
+-----[SHA256]-----+

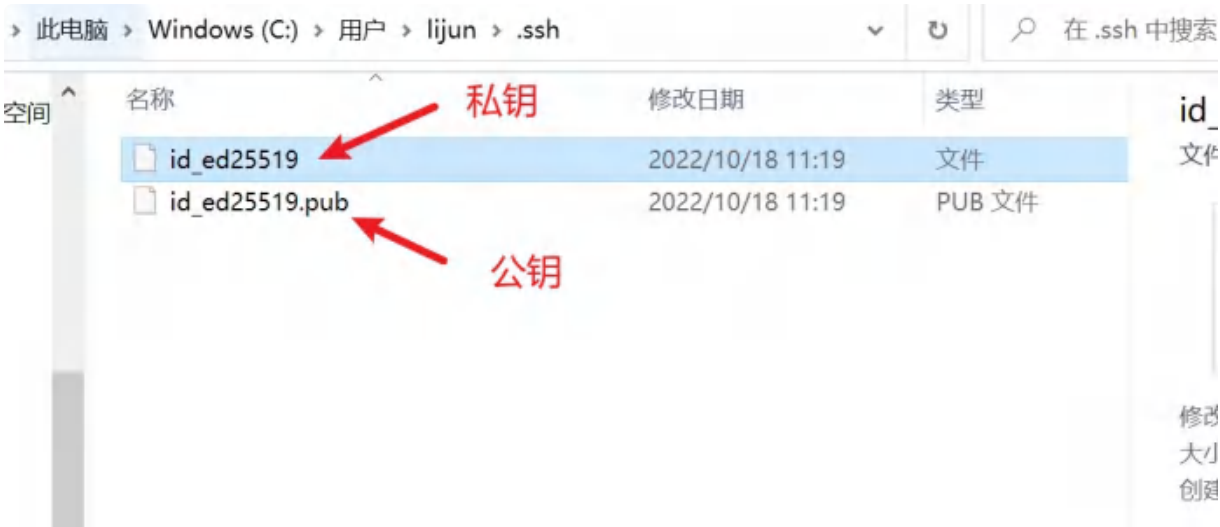
```

```

lijunjie@DESKTOP-K6RFVC1 MINGW64 ~/Desktop

```

在路径下, 打到 公/私钥



打开 .pub 文件, 复制内容, 粘贴到公钥

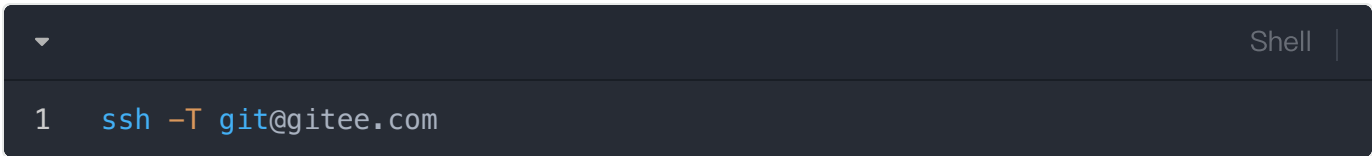


结果如下

您当前的SSH公钥数: 1



在添加完公钥后, 输入命令



```

+--[ED25519 256]--+
|..oo.|
|+ o oo =|
|+ o. * + +|
|+ . . o o + X . .|
|+ . . +So % + o.|
|+ . . Eo+.o + + .|
|+ . . X. . . .|
|+ . . = + .|
|+ . . .o. . .|
+-----[SHA256]-----+

liumjie@DESKTOP-K6RFVC1 MINGW64 ~/Desktop
$ ssh -T git@gitee.com
The authenticity of host 'gitee.com (212.64.63.190)' can't be established.
ED25519 key fingerprint is SHA256:+ULzij2u99B9ewYFTw1Q4ErYG/aepHLbu96PAUCov88.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'gitee.com' (ED25519) to the list of known hosts.

```

iii. 推送到远程仓库

将已经存在的代码推到远程仓库



新建仓库

在其他网站已经有仓库了吗? [点击导入](#)

仓库名称 *   输入仓库的名称

test

归属  路径 *  test

仓库地址: 

仓库介绍 0/100

用简短的语言来描述一下吧

☐ 开源 (所有人可见) 

☒ 私有 (仅仓库成员可见)

☐ 企业内部开源 (仅企业成员可见) 

☐ 初始化仓库 (设置语言、.gitignore、开源许可证)

☐ 设置模板 (添加 Readme、Issue、Pull Request 模板文件)

点击 [创建](#)

推送本地项目到远程空仓库

将本地已有的仓库推送到远程

```
1 // git remote add origin 远程仓库地址
2 git remote add origin git@gitee.com:leerolee/test.git
3 git push -u origin "master"
```

- remote: 操作远程仓库. add 添加. origin=git@gitee.com:leerolee/test.git
- push: 推送, 将本地的代码上传到远程

iv. 从远程仓库克隆

新建仓库

在其他网站已经有仓库了吗? [点击导入](#)

仓库名称 * ✓

test1

归属



路径 * ✓

test1

仓库地址: <https://gitee.com/leerolee/test1>

仓库介绍

0/100

用简短的语言来描述一下吧

仓库地址: <https://gitee.com/leerolee/test1>

仓库介绍

0/100

用简短的语言来描述一下吧

☐ 开源 (所有人可见) ②

☒ 私有 (仅仓库成员可见)

☐ 企业内部开源 (仅企业成员可见) ②

☒ 初始化仓库 (设置语言、.gitignore、开源许可证)

选择语言

JavaScript

添加 .gitignore

Node

添加开源许可证

许可证向导 ②

请选择开源许可证

☒ 设置模板 (添加 Readme、Issue、Pull Request 模板文件)

☒ Readme 文件

☐ Issue 模板文件 ②

☐ Pull Request 模板文件 ②

添加README文件

☐ 选择分支模型 (仓库创建后将根据所选模型创建分支)

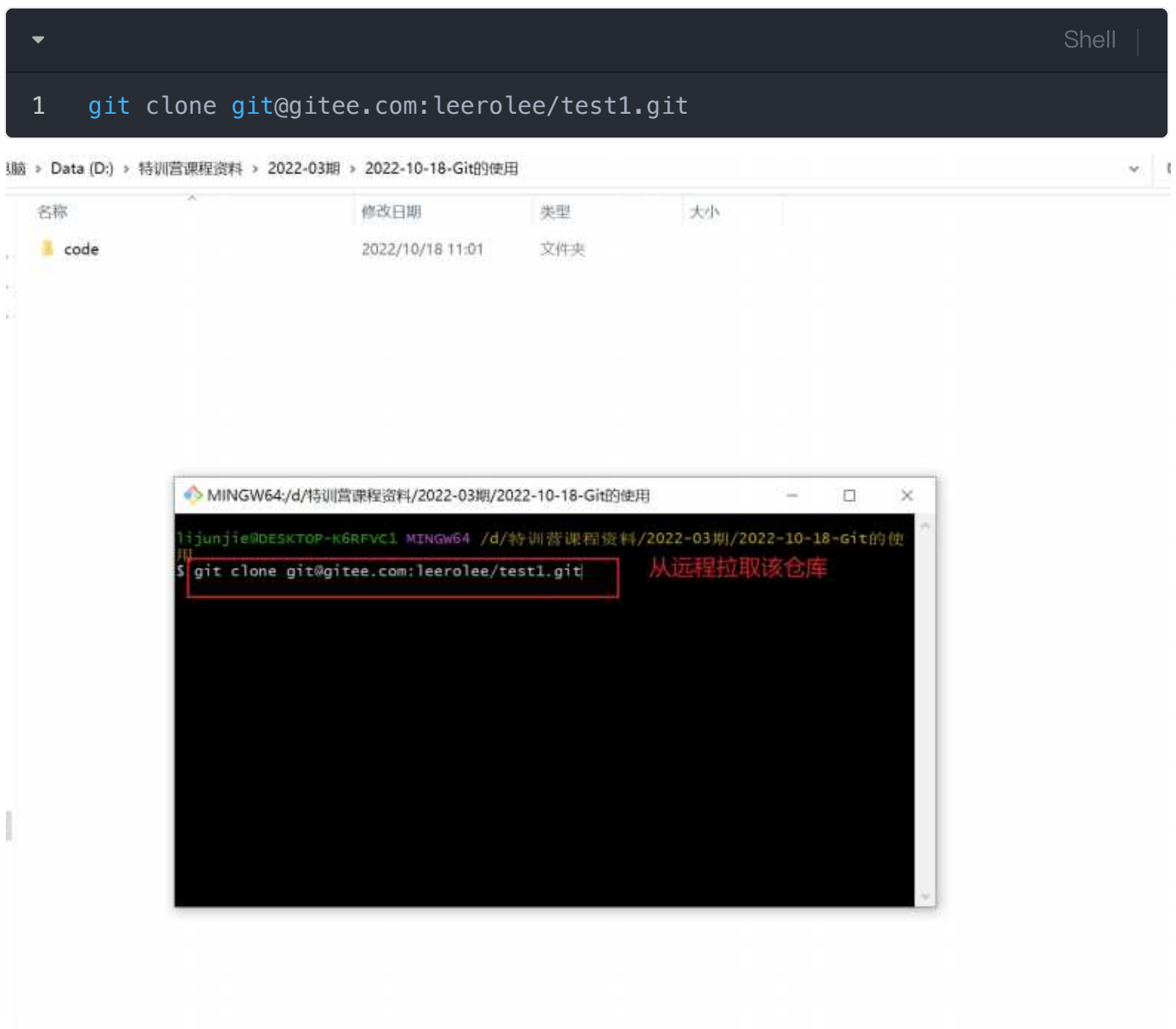
创建

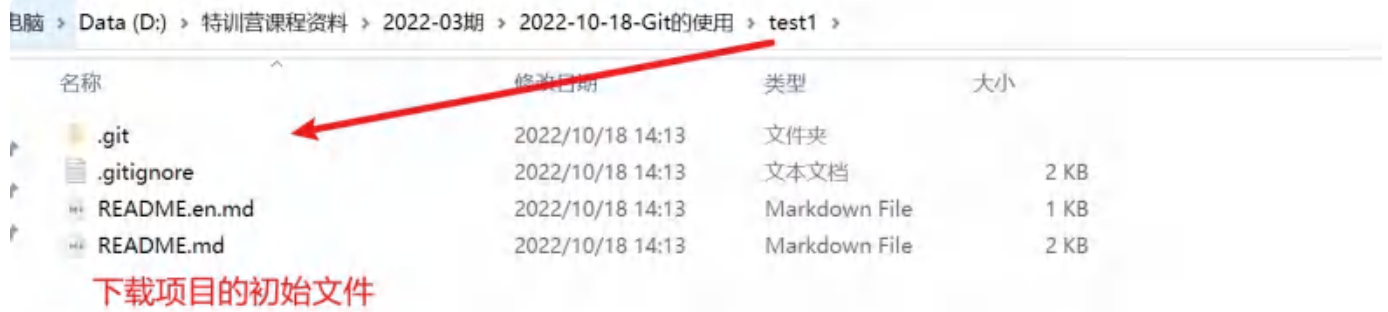
哪些文件不要提交到远程仓库

从远程仓库clone到本地



在目录下, 执行





创建代码, 编写代码

第一步

```
1 npm init -y
```

第二步

```
1 npm install vue
```

第三步

创建 src/index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta http-equiv="X-UA-Compatible" content="IE=edge" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>Document</title>
8   </head>
9   <body>
10     index
11   </body>
12 </html>
```

第四步

添加到index暂存区

```
▼ Shell |
1 git add .
```

第五步

提交到本地仓库

```
▼ Shell |
1 git commit -m '1-初始化'
```

重复, 第四步, 第五步 . 在本地进行多个版本

在需要推送的远程仓库时(下班之前, 在需要整合代码时), 执行

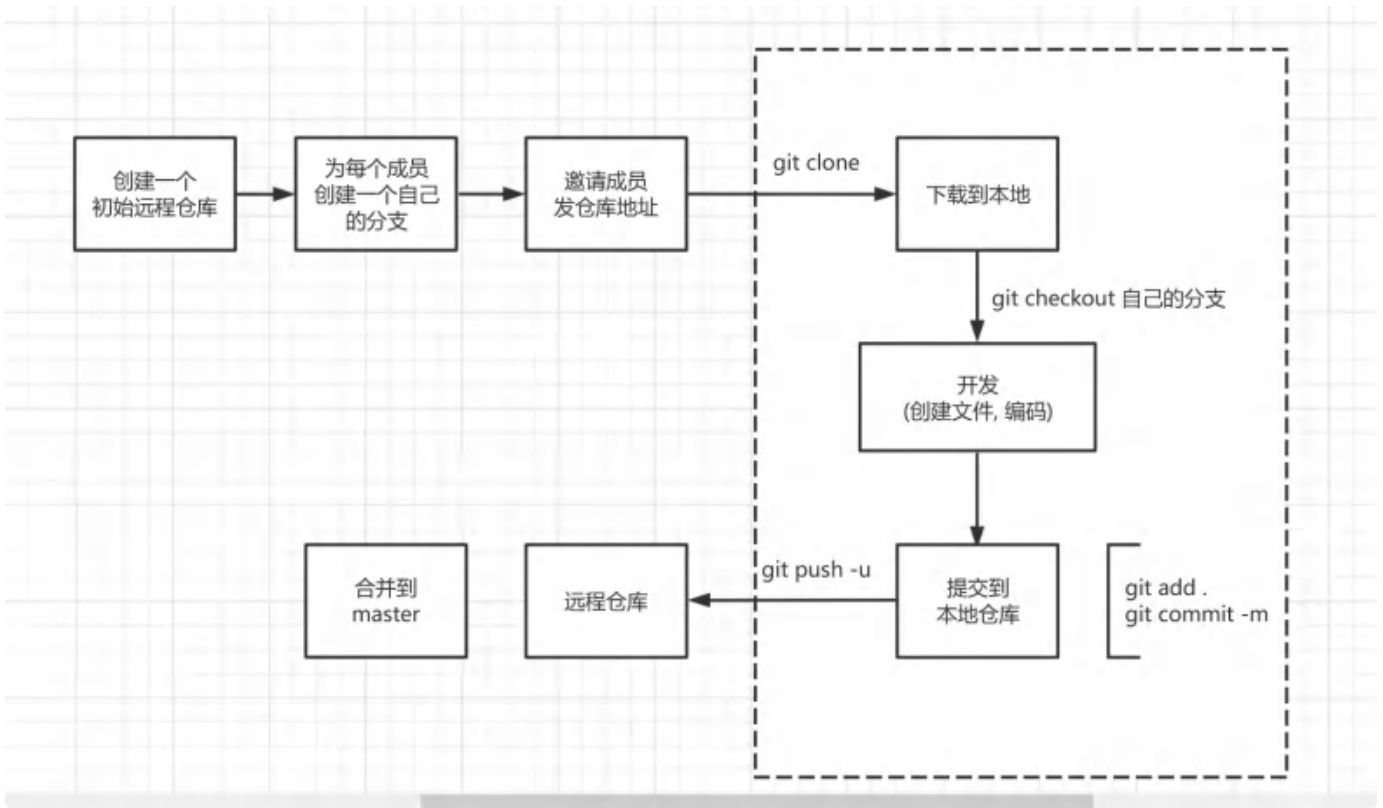
```
▼ Shell |
1 git push
```

如果远程仓库的版本跟本地仓库不同步. 远程仓库中, 如果有其它的提交, 会比自己的本地更新

此时, 需要先通过 `git pull` 命令将新的修改同步到本地

```
▼ Shell |
1 git pull
```

6. 实际开发项目经验——多人协作开发



※如果远程与本地仓库不同步，需要使用git pull拉回到本地

这个地方非常重要，在某一个版本进行开发，开发一个新的分支，而不去影响运行稳定的主分支，检查无问题后再合并回主分支。用于多成员协作，也可以用在出现分歧时，通过分支来校验bugfix。

一般而言，组员不可以在主分支上直接提交版本，各分支合并回主分支的操作也是组长操作

a. 小组成员本地开发

①结合组长的gitee码克隆到本地

git clone gitee码

②切换到自己的分支

git checkout name

③开发后提交到本地仓库，推送到远程

b. 组长

①切换到成员分支，并拉回本地，检查开发情况

②切换到主分支，合并成员分支到主分支，推送到远程